CSL 7640: NATURAL LANGUAGE UNDERSTANDING

Assignment- 2

Name: Shruti Chaudhary
Roll no.: B22AI037

PROBLEM 1: LEARNING WORD EMBEDDINGS FROM IIT JODHPUR DATA

Task 1: Dataset Preparation

Data Collection

The textual dataset was collected from multiple IIT Jodhpur-related sources to capture domain-specific academic vocabulary. The following sources were used:

- IIT Jodhpur postgraduate program webpage
- Faculty webpage (Prof. Anand Mishra website)
- Academic regulation (PDF files)
- Circulars (PDF files)

The webpages were scraped using Python libraries such as requests and BeautifulSoup, while the academic documents were processed using pdfplumber to extract textual content.

All extracted text was stored as separate documents and combined to form the raw corpus.

Preprocessing Steps

The collected textual data was cleaned and normalized using the following preprocessing pipeline:

- Lowercasing
- All text was converted to lowercase to maintain uniformity.
- Removal of Degree Variations
    - Different forms of academic degrees were standardized:
    - B.Tech → btech
    - M.Tech → mtech

    - M.Sc → msc
    - Ph.D → phd
- Removal of URLs
- Removal of Numbers
- Removal of Punctuation
- Removal of Extra Spaces
- Multiple spaces were replaced with a single space.
- Tokenization.
- Filtering Non-alphabetic Tokens
- Words with length less than 3 characters and greater than 15 characters were removed.
- Stopword Removal

After preprocessing, each document was converted into a list of tokens forming the final clean corpus.

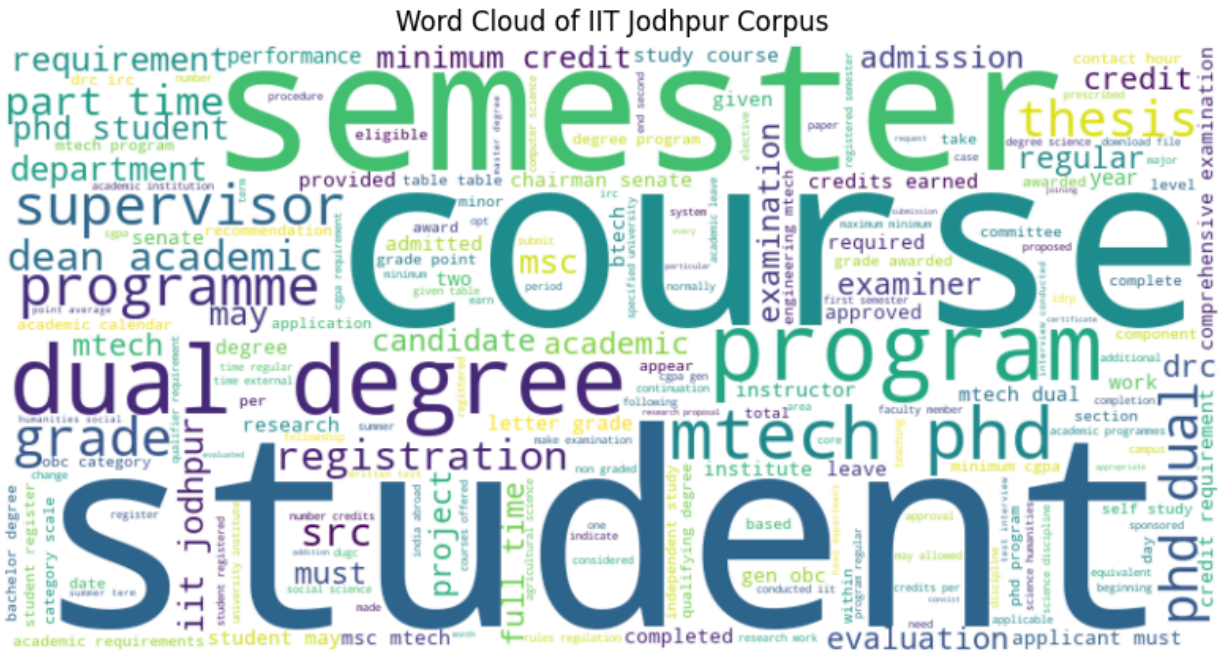
Dataset Statistics:

Total Documents	4
Total tokens	15077
Vocabulary size	1736

Top 10 most frequent Words:

('student', 358), ('course', 258), ('mtech', 252), ('phd', 234), ('degree', 208), ('semester', 186), ('students', 180), ('academic', 174), ('program', 166), ('requirements', 143)

Word Cloud


Word Cloud of IIT Jodhpur Corpus
student
course
semester
program
degree
dual
mtech
phd
registration
supervisor
dean academic
programme
grade
thesis
credit
minimum credit
requirement
performance
admission
part time
phd student
department
regular
examiner
examination
chairman senate
credits earned
required
approved
candidate
academic
instructor
mtech dual
drc
dual degree
iit jodhpur
project
full time
evaluation
applicant must
msc mtech
student may
completed
institute leave
minimum cgpa
gen obc
must
src
research
letter grade
msc
admitted
provided
table table
study course
given
degree program
eligible
comprehensive examination
credit requirement
phd dual
work
section
component
complete
committee
grade awarded
year
level
academic requirements
bachelor degree
student register
category scale
obc category
date
number credits
qualifying degree
independent study
within
based
considered
one indicate
conducted iit
program regular
may allowed
credits per
phd program
science humanities
science discipline
applicable
need
equivalent
beginning
day
sponsored
self study
test interview
approval
discipline
research work
rules regulation
social science
agricultural science
india abroad
courses offered
university institute
student registered
qualifier requirement
time regular
time external
per
humanities social
summer
fellowship
make examination
non graded
written test
evaluated
change
appear
cgpa gen
continuation
following
research proposal
area
total
faculty member
core
teaching
appropriate
campus
academic programmes
completion
additional
certificate
idrp
interview
conducted
maximum minimum
submission
every
particular
normally
specified university
academic leave
engineering mtech
system
minor
opt
award
grade point
minimum
two
given table
earn
period
submit
btech
cgpa requirement
recommendation
senate
point average
academic calendar
application
may
mtech
degree
drc irc
number
procedure
mtech program
academic institution
term
contact hour
prescribed
degree science
download file
major
joining
awarded
take
case
request
paper
registered semester
elective
master degree
computer science
second
degree
level
year
regular
credit
thesis
given
admission
credit
minimum credit
study course
performance
requirement
part time
phd student
department
supervisor
dean academic
programme
grade
student
course
semester
program
degree
dual
mtech
phd
registration

```
drc  (0.3365)
opt  (0.3324)

Top 5 nearest neighbors of the word: exam
lectures  (0.4291)
deems  (0.2855)
worth  (0.2850)
endorsement  (0.2674)
theendof  (0.2670)
```

The neighbors for **research** are weakly related semantically. Words such as monitor and changes loosely relate to research activities, but others like stay and branch are less meaningful. This suggests that the corpus size may be limited, leading to weaker semantic structure for this term.
The word students and admissions are semantically meaningful neighbors for student. However, words such as world and approved are less directly related. This indicates moderate semantic capture for academic context.
The neighbors for **phd** are semantically meaningful. Words such as msc and degree are strongly related to academic qualifications. This shows that the model successfully captures educational hierarchy relationships.

Analogy Experiment

```
student : exam :: professor : ?
lectures 0.37
deems 0.32
national 0.31
attends 0.30
resubmit 0.29
```

The neighbor lectures is semantically relevant to exam, but other words are weakly related. This suggests limited contextual learning due to small corpus size.

```
course : exam :: research : ?
supplementary 0.31
lectures 0.29
type 0.28
absent 0.27
worth 0.26
```

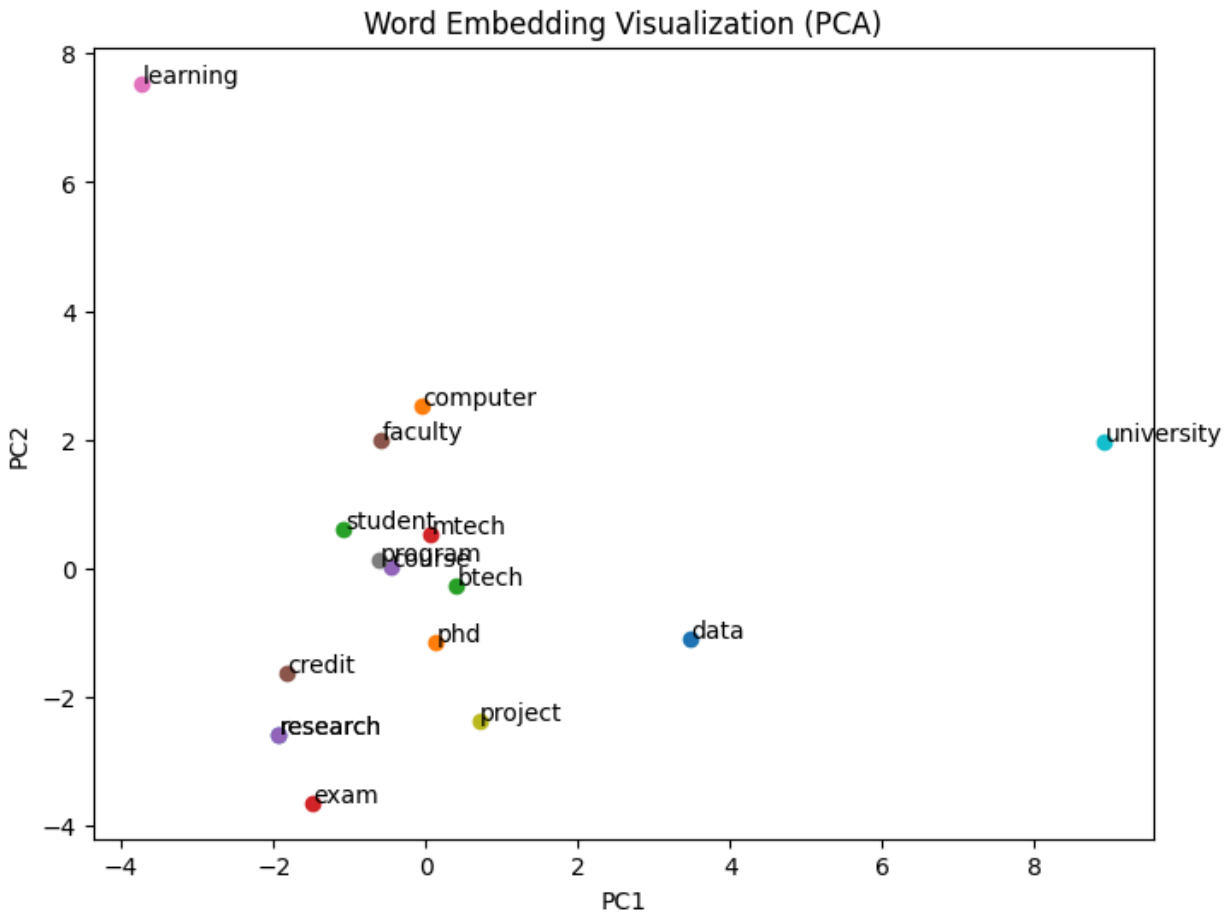
The result lectures is semantically meaningful because professors typically deliver lectures while students take exams. This indicates that the model captures role-based relationships.

```
semester : course :: degree : ?
phd 0.43
msc 0.37
determine 0.31
```

certified 0.30 unsatisfactory 0.28

The results are not strongly meaningful. While *supplementary* loosely relates to exams, the analogy does not clearly reflect a semantic relationship. This suggests that the model struggles with more complex analogies.

Task4: Word Embedding Visualization using PCA (CBOW)



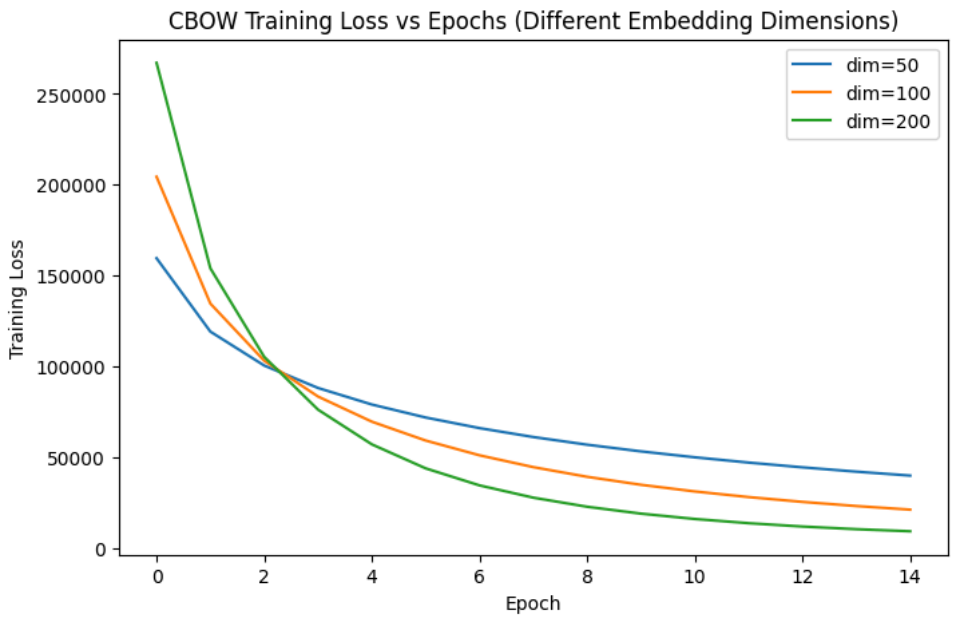
Task2: Model Training

Effect of Embedding dimension on CBOW:

To study the impact of embedding dimensionality, CBOW models were trained with embedding sizes **50, 100, and 200**. The training loss and semantic quality of embeddings were compared.

Embedding Dimension	Final Loss

50	39849.67
100	21094.50
200	9179.66



As embedding dimension increases, the model capacity improves, allowing it to capture richer semantic relationships. The lowest loss is obtained for embedding dimension 200.

Semantic Quality Comparison

Embedding dim = 50

Nearest to 'btech':
[('semesters', 0.44), ('sanction', 0.43), ('engineering', 0.43), ('ongoing', 0.41), ('dual', 0.40)]

Nearest to 'professor':
[('decided', 0.44), ('sabbatical', 0.44), ('separately', 0.42), ('student', 0.389), ('dean', 0.38)]

Embedding dim = 100

Nearest to 'btech':
[('compulsory', 0.4095030601848361), ('submit', 0.3284356753087475), ('appeal', 0.32267988370619927), ('submission', 0.32143264170482), ('phd', 0.3213245643315113)]

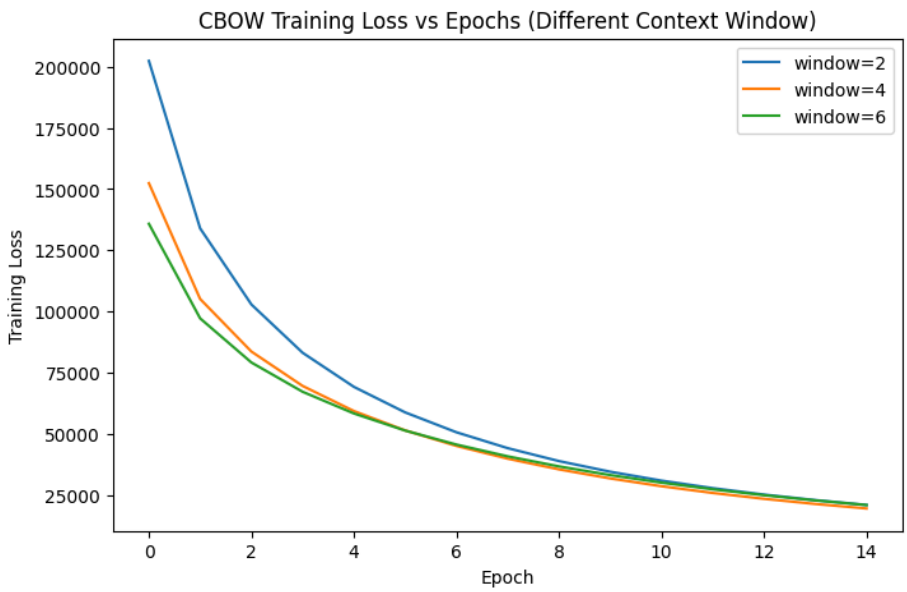
Nearest to 'professor':
[('breaks', 0.28087304947797226), ('head', 0.27685655798926095), ('availability', 0.27521669061169995), ('nbhm', 0.26807495740021964), ('review', 0.2661445839263668)]

```
semester : course :: degree : ?
phd 0.43
msc 0.37
determine 0.31
certified 0.30
unsatisfactory 0.28
```

Effect of Window size on CBOW

Window Size	Final Loss
2	20991.21
4	19545.51
6	20948.32

The model with window size **4** achieved the lowest loss, indicating better learning when moderate contextual information is used.



Semantic Quality Comparison

```
Window size = 2

Nearest to 'btech':
[('discussion', 0.36), ('would', 0.34), ('duration', 0.33), ('gap', 0.31), ('msc', 0.30)]
```

```
Nearest to 'professor':
[('curricula', 0.37), ('pass', 0.30), ('registers', 0.29), ('evaluated', 0.29), ('motion', 0.28)]
```

With a small window size, the model captures local context only. Some neighbors like msc and curricula are meaningful, but several unrelated words appear. The semantic structure is weak.

```
Window size = 4

Nearest to 'btech':
[('weekends', 0.38), ('academic', 0.38), ('semesters', 0.35), ('endeavors', 0.35), ('directives',
0.33)]

Nearest to 'professor':
[('requested', 0.35), ('postdoctoral', 0.33), ('use', 0.32), ('curricula', 0.28), ('sme', 0.28)]
```

Window size 4 provides more meaningful academic relationships. Words such as academic, semesters, and postdoctoral are semantically related. This indicates improved contextual understanding.

```
Window size = 6

Nearest to 'btech':
[('commencement', 0.39), ('lecture', 0.32), ('conditions', 0.31), ('financial', 0.315), ('full', 0.30)]

Nearest to 'professor':
[('yet', 0.29), ('rule', 0.29), ('received', 0.27), ('departmental', 0.27), ('assignments', 0.26)]
```

With a large window size, the model captures broader context but introduces noise. Some words like *lecture* and *departmental* are relevant, but many neighbors are weakly related.

Task3: Semantic Analysis (SkipGram Model)

Nearest Neighbors using cosine similarity

```
Using skip gram embeddings

Top 5 nearest neighbors of the word: research
work: 0.4328
presentation: 0.4305
structures: 0.4263
sum: 0.4152
cleared: 0.4086

Top 5 nearest neighbors of the word: student
may: 0.5068
```

```
alone: 0.4769
admitted: 0.4438
required: 0.4415
allowed: 0.4279

Top 5 nearest neighbors of the word: phd
mtech: 0.6040
degree: 0.5810
semesters: 0.4777
points: 0.4752
dual: 0.4317

Top 5 nearest neighbors of the word: exam
absence: 0.4409
agricultural: 0.4227
intersection: 0.4124
suggested: 0.4117
computer: 0.3870
```

The neighbors work and presentation are semantically meaningful in academic research context. However, words such as sum and cleared are less related. The model partially captures research-related semantics.
Words like admitted, required, and allowed relate to student regulations. However, words such as may and alone are weakly related, indicating limited contextual understanding.
The neighbors are semantically meaningful. Words such as mtech and degree correctly represent academic qualifications. This indicates that Skip-Gram effectively captures educational relationships.
The word *absence* is related to exam context, but other neighbors are weakly connected. This suggests limited semantic learning for this term.

Analogy Experiment (Skipgram Model)

```
student : exam :: professor : ?
absence 0.48
agricultural 0.480
chosen 0.44
functioning 0.436
act 0.43
```

The results are not semantically meaningful. The model fails to capture the relationship between student-exam and professor-related activities.

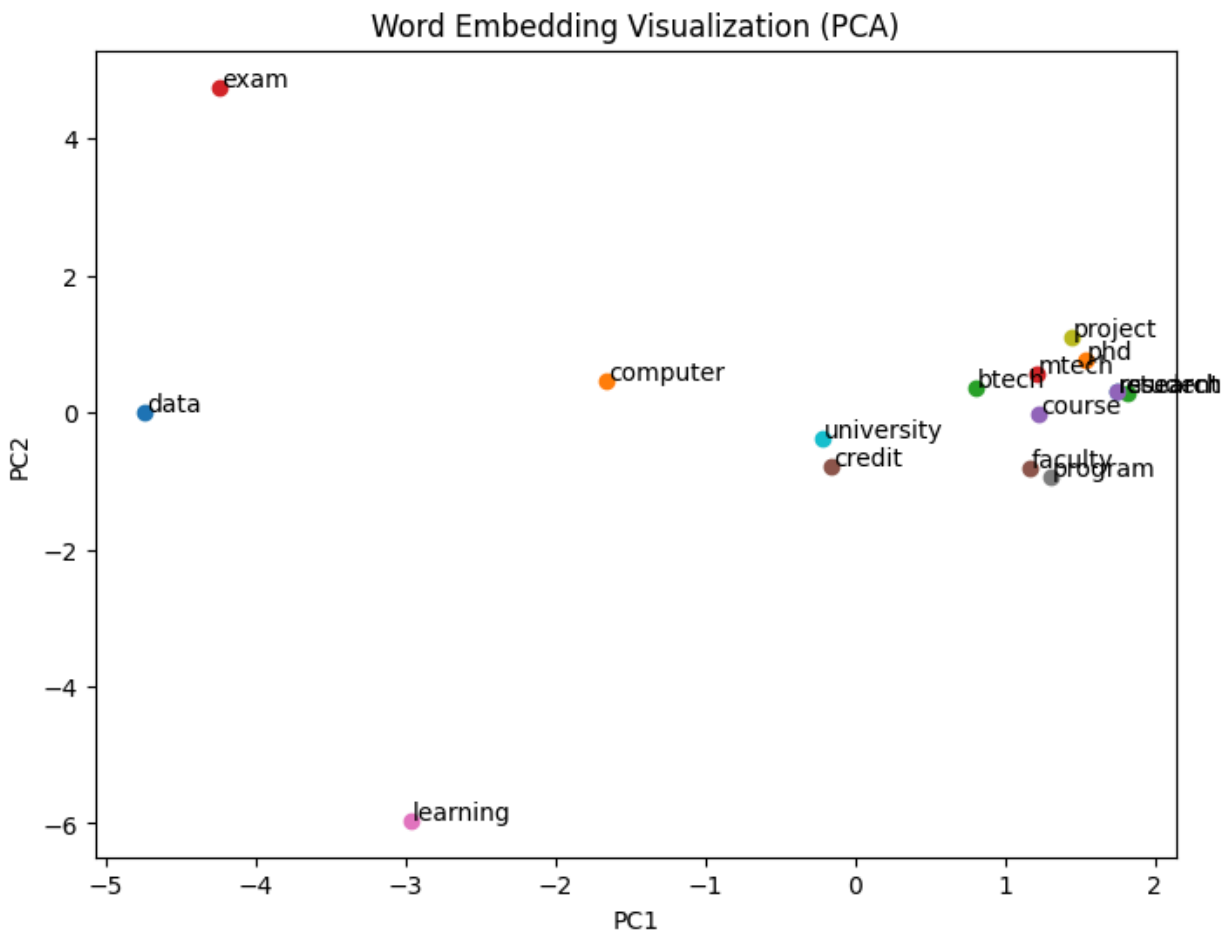
```
course : exam :: research : ?
areas 0.412
intersection 0.388
going 0.387
absence 0.379
appointment 0.375
```

The predicted words are weakly related. The model struggles with this analogy, indicating limited ability to capture higher-level semantic relationships.

```
semester : course :: degree : ?
dual 0.520
mtech 0.453
engineering 0.452
governors 0.429
automatic 0.410
```

The predictions dual and mtech are meaningful, as they correspond to types of degree programs. This indicates that Skip-Gram captures hierarchical academic relationships.

Task4: Word Embedding Visualization using PCA (SkipGram)

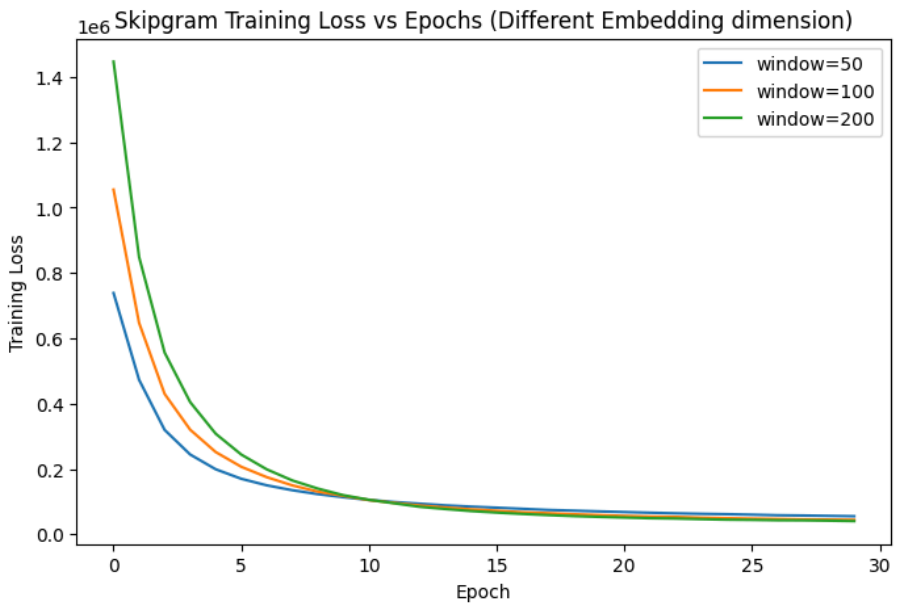


Task2: Model Training

Effect of Embedding dimension on Skip-Gram:

To study the impact of embedding dimensionality, Skip-Gram models were trained with embedding sizes 50, 100, and 200. The training loss and semantic quality of embeddings were analyzed.

Embedding Dimension	Final Loss
50	55456.41
100	44412.39
200	40606.49



The training loss decreases as embedding dimension increases. This indicates that higher dimensional embeddings provide greater representational capacity for modeling word relationships.

Semantic Quality Comparison

Skipgram embedding dim = 50

Nearest to 'btech':
[('programmes', 0.63), ('undergraduate', 0.633), ('discipline', 0.627), ('compulsory', 0.621), ('program', 0.619)]

Nearest to 'research':

```
[('field', 0.610), ('requisite', 0.592), ('proposal', 0.580), ('indiscipline', 0.578), ('bachelors', 0.522)]
```

The semantic relationships are strong and meaningful. Words such as undergraduate, program, and proposal are closely related to btech and research. This embedding dimension captures domain-specific context effectively.

```
Skipgram embedding dim = 100

Nearest to 'btech':
[('program', 0.510), ('msc', 0.477), ('duration', 0.474), ('admissions', 0.449), ('extension', 0.442)]

Nearest to 'research':
[('proposal', 0.449), ('slow', 0.422), ('takes', 0.414), ('qualifier', 0.414), ('wide', 0.411)]
```

The semantic quality is still reasonable, but slightly weaker than dimension 50. Some unrelated words like slow and takes appear, indicating reduced semantic coherence.

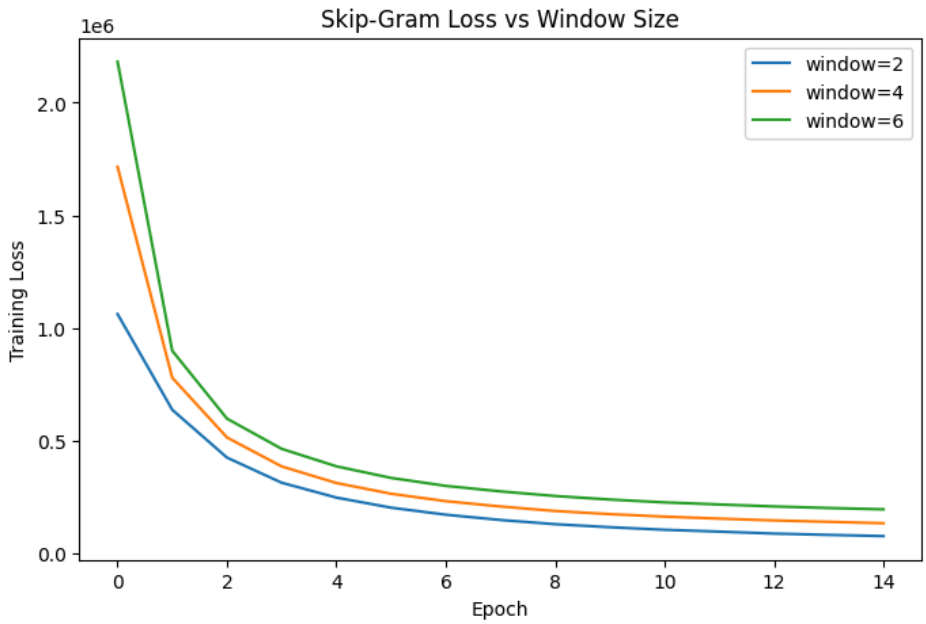
```
Skipgram embedding dim = 200

Nearest to 'btech':
[('program', 0.41), ('mtech', 0.347), ('interview', 0.340), ('msc', 0.335), ('date', 0.319)]

Nearest to 'research':
[('bvsc', 0.340), ('fresh', 0.321), ('provisional', 0.312), ('limited', 0.308), ('members', 0.306)]
```

Although the loss is lowest, semantic quality deteriorates. Several unrelated words appear, suggesting that higher embedding dimensions may lead to overfitting when the dataset size is limited.

Effect of Window size on Skip - Gram

Window Size	Final Loss
2	76426.51
4	133330.15
6	195189.91

The lowest loss is achieved with window size = 2, while increasing the window size leads to higher loss. This is because Skip-Gram becomes more difficult to optimize as the context window increases.



Semantic Quality Comparison

Skipgram Window size = 2

Nearest to 'btech':
[('discipline', 0.599), ('programme', 0.55), ('engineering', 0.528), ('specialization', 0.518), ('student', 0.505)]

Nearest to 'professor':
[('seminar', 0.462), ('equal', 0.4616), ('examiners', 0.456), ('level', 0.438), ('category', 0.434)]

The model captures strong local relationships. Words like engineering, programme, and student are highly relevant to btech. This setting produces clear and meaningful semantic relationships

Skipgram Window size = 4

Nearest to 'btech':
[('mtech', 0.553), ('msc', 0.533), ('program', 0.520), ('department', 0.512), ('cse', 0.509)]

Nearest to 'professor':
[('recommendation', 0.484), ('associate', 0.482), ('quantity', 0.472), ('running', 0.461), ('supervising', 0.450)]

The semantic quality improves further. Words like mtech, msc, and department indicate strong academic relationships. For professor, terms like associate and supervising are highly meaningful. This window size captures both local and broader context effectively.

Skipgram Window size = 6

Nearest to 'btech':
[('dual', 0.582), ('msc', 0.581), ('specialization', 0.525), ('mtech', 0.514), ('major', 0.510)]

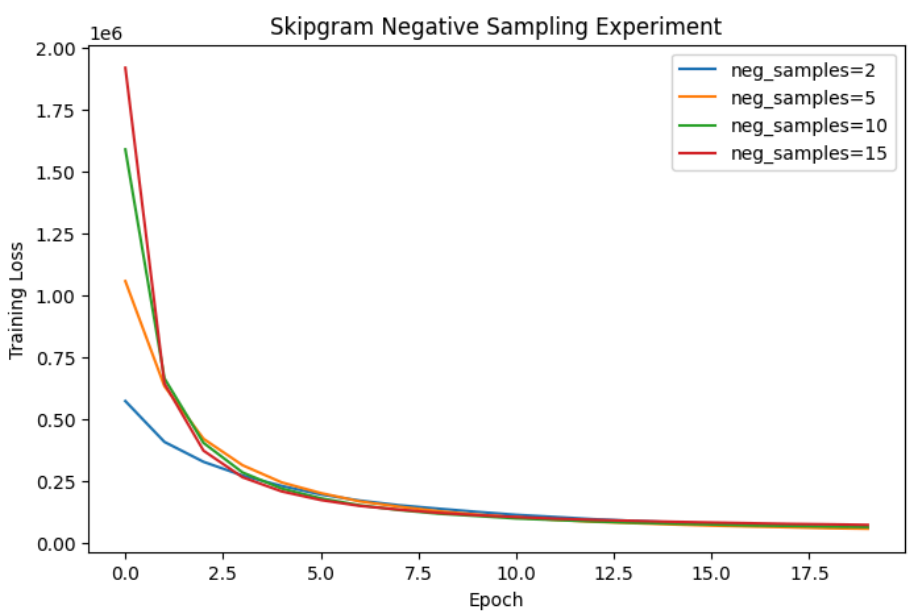
Nearest to 'professor':
[('contention', 0.506), ('nominated', 0.483), ('supervising', 0.47), ('committee', 0.469), ('resignation', 0.469)]

The model captures broader academic context. Words like dual, major, and committee are meaningful. However, some noise appears due to inclusion of distant context.

Effect of number of negative samples:

Number of negative samples	Final Loss
2	61876.95
5	58911.00
10	66281.03
15	74693.45

Using 5 negative samples gives the lowest loss. Increasing negative samples beyond this increases computational complexity and does not improve convergence.



Semantic Quality Comparison

```
Negative samples = 2

Nearest to 'btech':
[('programme', 0.591), ('assistance', 0.511), ('conditions', 0.500), ('minimum', 0.484),
('program', 0.481)]

Nearest to 'research':
[('src', 0.488), ('drc', 0.449), ('program', 0.445), ('two', 0.410), ('candidacy', 0.409)]
```

The relationships are weak. Some words such as *programme* are meaningful, but others like *two* are not semantically related. Small negative sampling leads to poor discrimination between words.

```
Negative samples = 5

Nearest to 'btech':
[('program', 0.498), ('dual', 0.494), ('engineering', 0.491), ('specialization', 0.4813), ('mtech',
0.472)]

Nearest to 'research':
[('src', 0.514), ('proposal', 0.510), ('fields', 0.459), ('upon', 0.452), ('composition', 0.443)]
```

Semantic quality improves significantly. Words like engineering, mtech, and proposal are meaningful. This configuration balances training efficiency and semantic representation.

```
Negative samples = 10

Nearest to 'btech':
[('programme', 0.584), ('mtech', 0.505), ('program', 0.499), ('engineering', 0.491), ('programs',
0.483)]

Nearest to 'research':
[('work', 0.208), ('obtaining', 0.444), ('related', 0.433), ('committee', 0.429), ('comparable',
0.425)]
```

Semantic quality remains strong, but improvements are marginal compared to 5 samples. Training cost increases.

```
Negative samples = 15

Nearest to 'btech':
[('programme', 0.567), ('program', 0.566), ('admitted', 0.521), ('mtech', 0.512), ('minimum',
0.504)]

Nearest to 'research':
```

```
[('proposal', 0.504), ('field', 0.495), ('src', 0.495), ('indicated', 0.481), ('assessment', 0.458)]
```

Semantic relationships remain reasonable but not significantly better. Larger negative samples increase training time without notable semantic improvement.

PROBLEM 2: CHARACTER-LEVEL NAME GENERATION USING RNN VARIANTS

Task 0: Dataset Preparation

To train the character-level name generation models, a dataset of 1000 Indian names was created using a Large Language Model (LLM).

The generated names were stored in a text file named TrainingNames.txt, where each line contains one name.

Task 1: Model Implementation

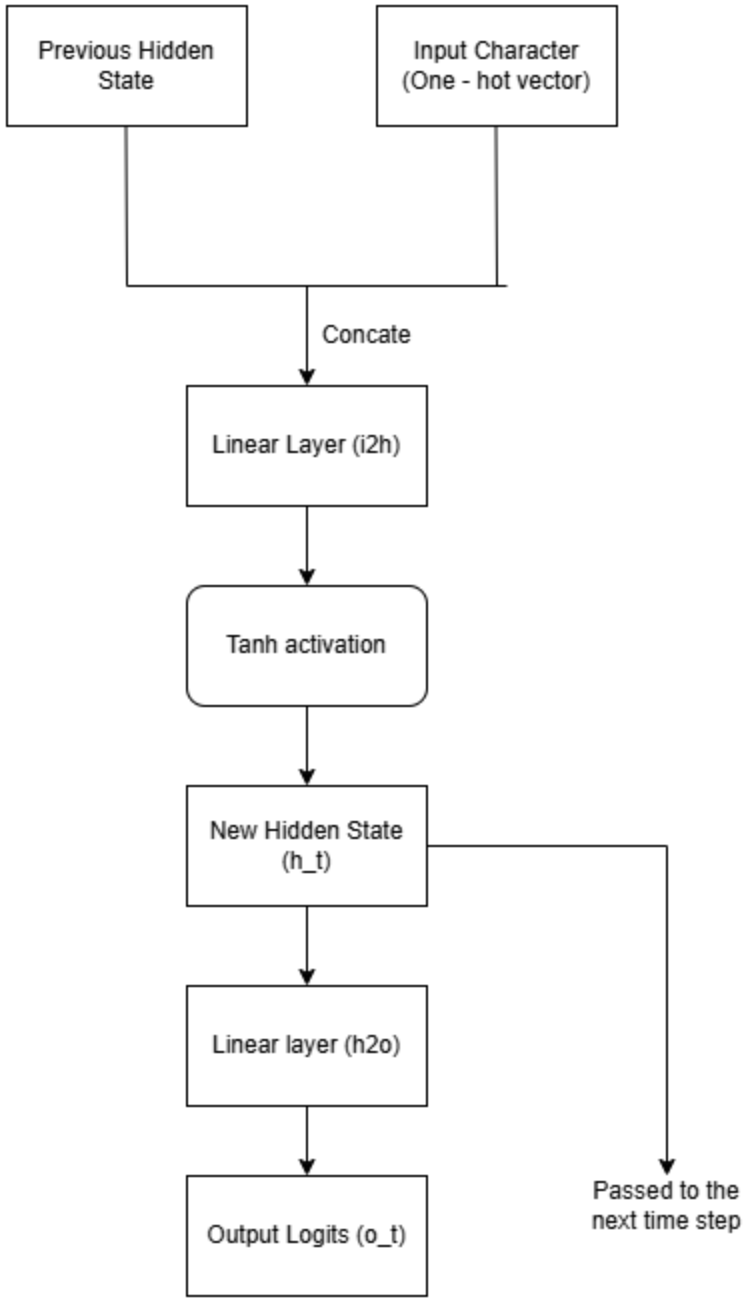
Vanilla Recurrent Neural Network (RNN)

The Vanilla RNN is a character-level recurrent neural network that predicts the next character in a sequence. The model processes one character at a time and maintains a hidden state that carries information from previous time steps.

Model Architecture

At each time step:

- The input is a one-hot encoded vector of size vocab_size(24)
- The input is concatenated with the previous hidden state
- The concatenated vector is passed through a linear layer followed by tanh activation to compute the new hidden state
- The hidden state is then passed through another linear layer to produce output logits
- The logits represent scores for the next character



Number of Trainable Parameters:

1.Input to hidden Layer:
Parameters

- Weights = (vocab_size + hidden_size) × hidden_size
- Bias = hidden_size

Total = (vocab_size+hidden_size)×hidden_size + hidden_size

2. Hidden to Output Layer:
Parameters:

- Weights = hidden_size × vocab_size
- Bias = vocab_size

Total = hidden_size×vocab_size + vocab_size

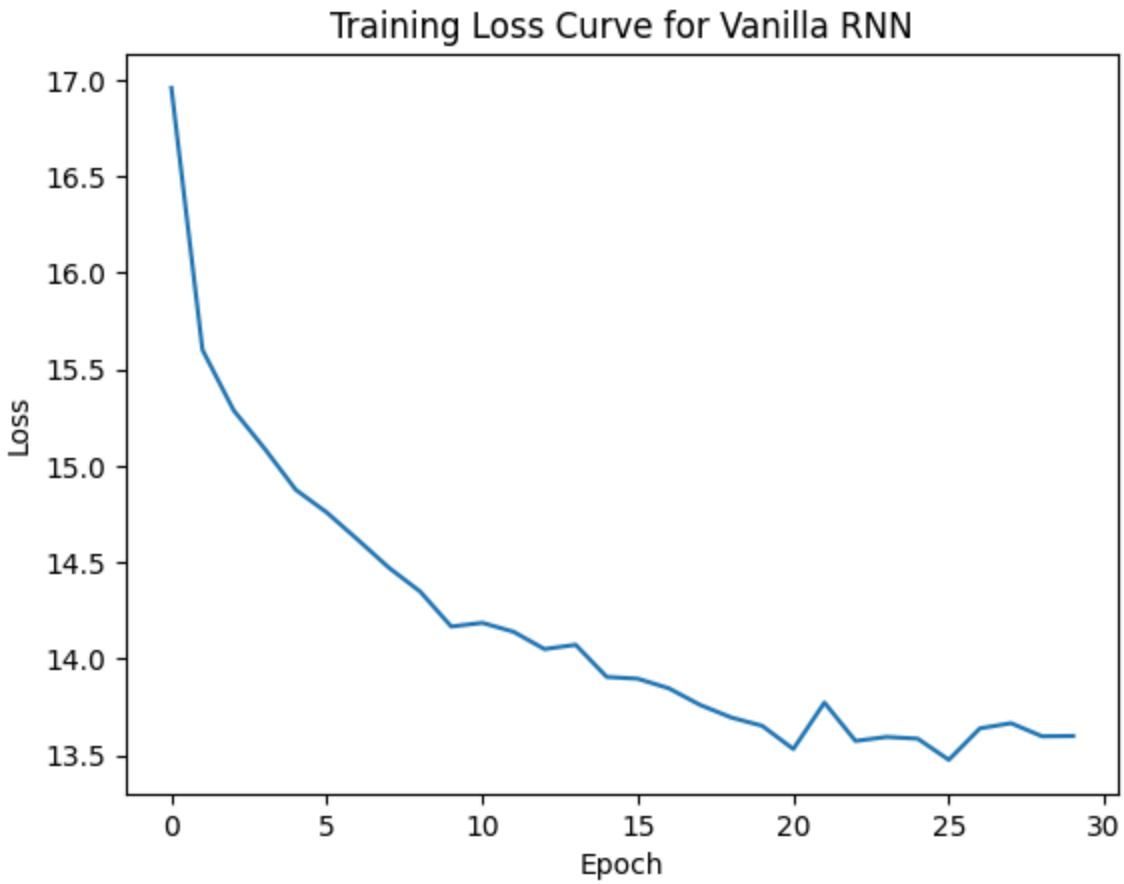
Total Trainable Parameters= (vocab_size+hidden_size)×hidden_size + hidden_size+hidden_size×vocab_size + vocab_size

vocab_size=25
hidden_size=128

Total trainable parameters= ((25 + 128) *128 + 128) + (128 *25)+25
= 22,937

Hyperparameters

Hyperparameters	Value
Hidden size	128
Number of layers	1
Learning rate	0.003
Optimizer	Adam
Number of epochs	30



Bidirectional Long Short-Term Memory (BLSTM)

Architecture Description

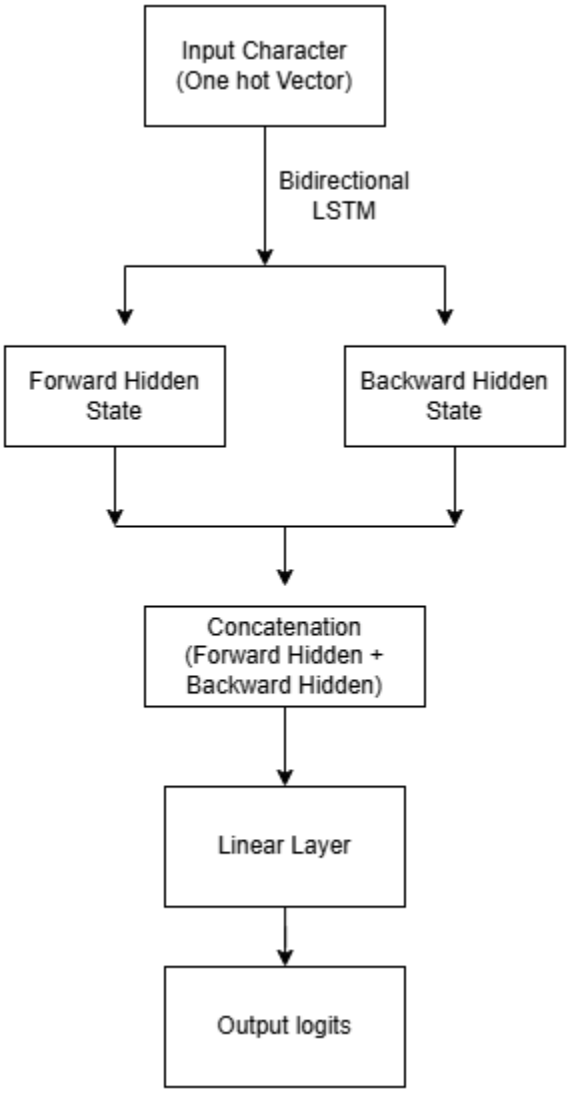
The Bidirectional LSTM (BLSTM) processes the input sequence in both forward and backward directions to capture past and future context simultaneously. Each character (one-hot encoded) is fed into a bidirectional LSTM layer, which produces two hidden states:

Forward hidden state (left → right)
Backward hidden state (right → left)

These two hidden states are concatenated and passed through a fully connected layer to produce output logits for next character prediction.
At each time step:

- Input character → one-hot vector
- Passed to Bidirectional LSTM
- Forward and backward hidden states computed
- Concatenate both hidden states

- Pass through linear layer
- Output logits generated



Number of Trainable Parameters:

1. Pytorch LSTM parameters per direction

For one LSTM: 4×(input_size×hidden_size+hidden_size×hidden_size+hidden_size)
For Bidirectional LSTM: 2x 4×(input_size×hidden_size+hidden_size×hidden_size+hidden_size)
= 8 x ((input_size×hidden_size+hidden_size×hidden_size+hidden_size)

2. Fully Connected Layer

Parameters:

- Weights= (2×hidden_size)×vocab_size
- Bias = vocab_size

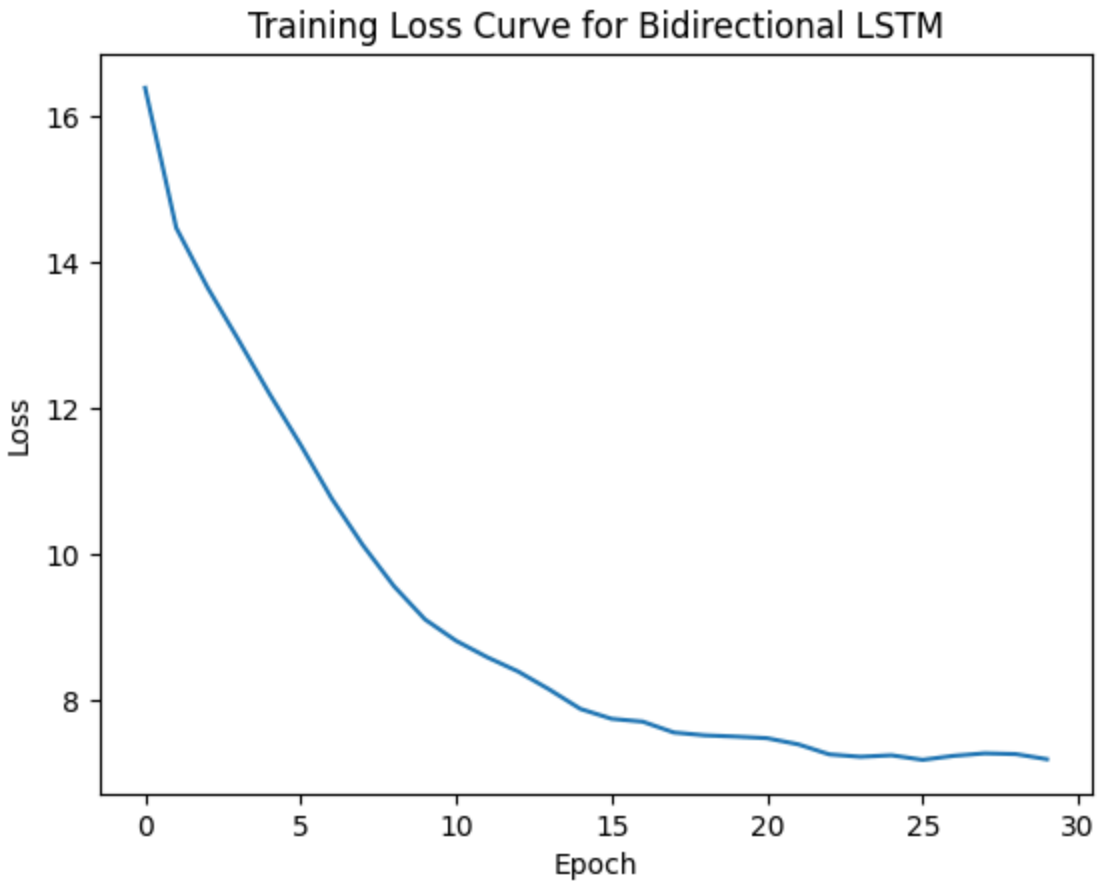
Total Trainable Parameters for BLSTM :
8(vocab_size · hidden_size+hidden_size^2+hidden_size)+(2 · hidden_size · vocab_size)+vocab_siz

vocab_size=25
hidden_size=128

Total trainable parameters= 164,121

Hyperparameters

Hyperparameters	Value
Hidden size	128
Number of layers	1
Learning rate	0.003
Optimizer	Adam
Number of epochs	30
Bidirectional	True



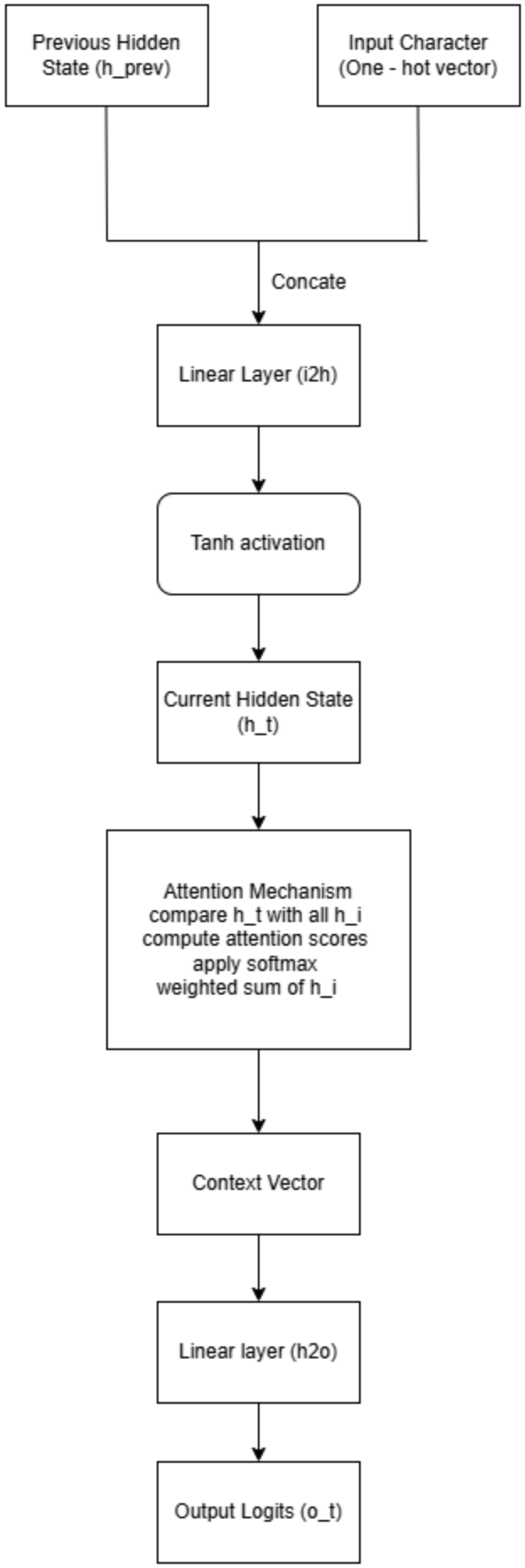
RNN with Basic Attention Mechanism

Architecture Description

The RNN with Attention extends the Vanilla RNN by allowing the model to focus on relevant previous hidden states when predicting the next character. Instead of relying only on the last hidden state, the model computes a context vector using attention weights over all past hidden states.

At each time step:

- Input character is converted to one-hot vector
- Concatenate input with previous hidden state
- Pass through linear layer + tanh → new hidden state
- Compute attention scores between current hidden and all previous hidden states
- Apply softmax to obtain attention weights
- Compute weighted sum of previous hidden states → context vector
- Pass context vector through linear layer
- Output logits for next character prediction


Previous Hidden State (h_prev)
Input Character (One - hot vector)
Concate
Linear Layer (i2h)
Tanh activation
Current Hidden State (h_t)
Attention Mechanism
compare h_t with all h_i
compute attention scores
apply softmax
weighted sum of h_i
Context Vector
Linear layer (h2o)
Output Logits (o_t)

Number of Trainable Parameters

1. Input-to-hidden layer
Parameters
- Weights: (vocab_size+hidden_size)×hidden_size
- Bias: hidden size

2. Attention Layer
Parameters:
- Weights: (2 x hidden_size) x1
- Bias: 2x hidden_size+1

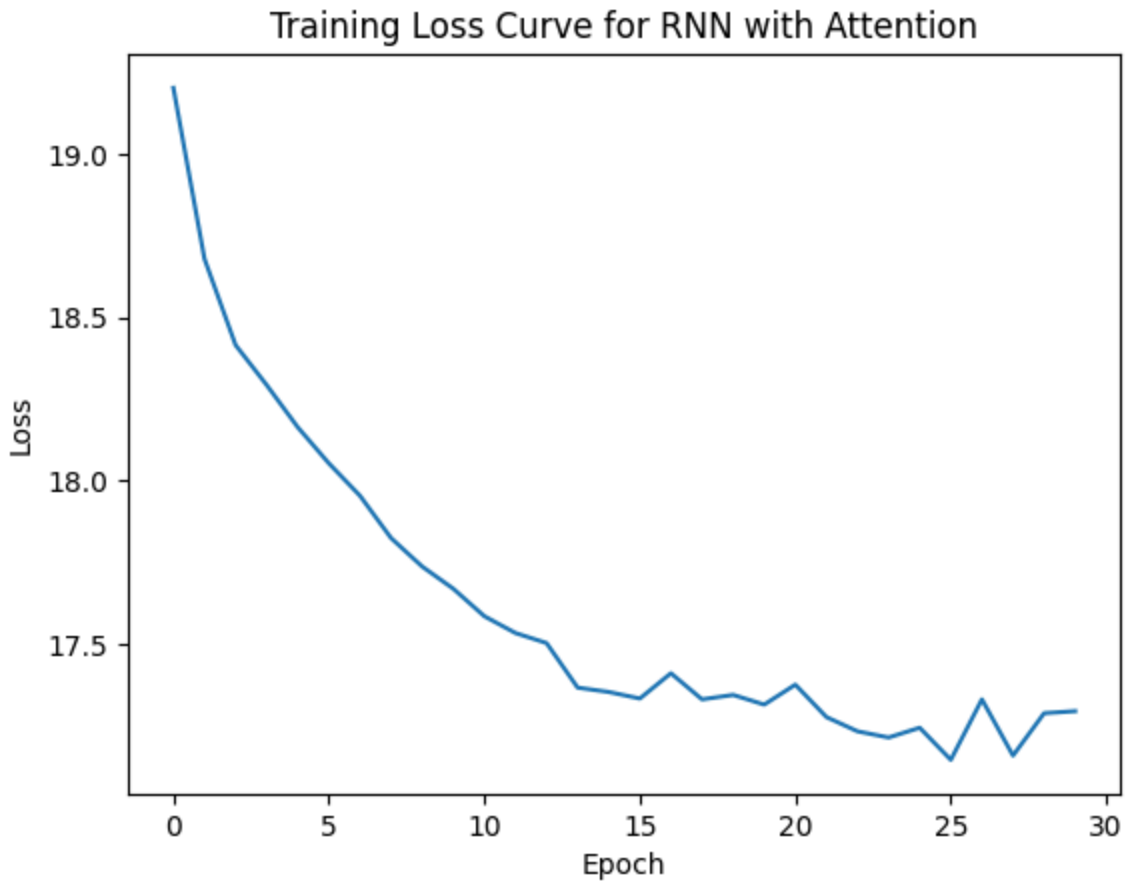
2. Output Layer
Parameters:
- Weights: hidden_size x vocab_size
- Bias: vocab_size

Total Trainable Parameters:
(vocab_size+hidden_size)×hidden_size + hidden_size + (2 x hidden_size) x1+ 2x hidden_size+1+ hidden_size x vocab_size + vocab_size

Vocab_size=25
Hidden_size=128
Total Trainable Parameters = 23,450

Hyperparameters

Hyperparameters	Value
Hidden size	128
Number of layers	1
Learning rate	0.003
Optimizer	Adam
Number of epochs	30
Bidirectional	True



Task 2 : Quantitative Evaluation

Model	Novelty Rate	Diversity
Vanilla Recurrent Neural Network (RNN)	95.54	0.92
Bidirectional Long Short-Term Memory (BLSTM)	59.81	0.505
RNN with Basic Attention Mechanism	99.79	0.989

Task 3: Qualitative Analysis: Realism of Generated Names and Common Failure Modes

1. Vanilla RNN

Representative Generated Samples

Seen-like samples

rahiti, girish, arush, darshant, harvi, sahyan

Novel samples

aksha, yuday, ganash, govish, sanila, vikra, anindra

Realism Discussion

The Vanilla RNN generates highly diverse names, as reflected by its high novelty (95.54) and diversity (0.92). Many generated names are distorted or unnatural, indicating that the model struggles to maintain coherent structure over longer sequences. The increase in novelty has come at the cost of reduced realism.

Common Failure Modes

- Broken phonetic structure:ganash, vikra
- Repetition or irregular patterns: hanshansha, faloshanka

2. Bidirectional LSTM (BLSTM)

Representative Generated Samples

Seen-like samples

riya, hriday, santosh, neeraj, gaurika, purushottam

Novel samples

greshma, devanga, aarhant, parkha, yaten, gaganveep

Realism Discussion

The BLSTM produces more structured and realistic names compared to the Vanilla RNN. Names such as riya, hriday, and santosh are highly realistic. The bidirectional architecture helps maintain better character continuity and pronunciation. However, the model exhibits moderate novelty (58.81) and diversity (0.505), meaning it tends to stay closer to training data and generates fewer creative variations.

Common Failure Modes

- Slight distortions: greshma, aarhant

- Unnatural combinations: gakatmesh, nalok
- Name truncation or incomplete structure: devanga,parkha
- Reduced diversity (many outputs similar to training data)

3. RNN with Attention

Representative Generated Samples

Seen-like samples

rahiti, girish, arush, darshant, harvi, sahyan

Novel samples

radhas, lakshamt, davar, aksish, suhaa, aruvam, faraz

Realism Discussion

The RNN with attention achieves the highest novelty (99.79) and diversity (0.989), producing highly varied and creative names. Some outputs resemble realistic names such as radhas and faraz, but many are distorted due to excessive variation. The attention mechanism improves context usage, but in this case, it leads to over-generation of novel patterns, reducing overall realism.

Common Failure Modes

- Highly distorted names: ojgwiba, ekhish
- Unnatural phonetic combinations: aruvam, cheerani
- Incomplete or abrupt endings: upsh, aitit
- Over-randomization due to high diversity
- Rare coherent but unusual names: suhaa, davar, faraz, aksish